ML Experiment Number 9

Student Name :Vidya Rajan Dharne

Batch :CS1

Roll Number :CS12

Aim

To implement Classification and Regression Trees (CART) using Python on the Palmer Penguins Dataset Extended.

Objectives

1. To understand the concept and working of the CART algorithm.
2. To implement Decision Tree Classification using Python.
3. To visualize the constructed Decision Tree structure and decision boundaries.
4. To analyze impurity measures such as Gini Index and Entropy.
5. To compare the decision boundary of a Decision Tree with a linear classification boundary (e.g., Logistic Regression).
6. To evaluate and compare the performance of both classifiers.

Relevant Theory

Classification and Regression Trees (CART)

CART is a non-parametric supervised machine learning algorithm used for both classification and regression tasks. It builds a binary decision tree by recursively splitting the dataset based on feature values that maximize information gain or minimize impurity.

A tree consists of:

- Root Node: The topmost node representing the entire dataset.
- Internal Nodes: Decision points based on feature splits.

- Leaf/Terminal Nodes: Output class labels or predictions.

Impurity MeasuresGini Index (Gini Impurity): Measures the frequency with which a randomly chosen element from the set would be incorrectly labeled.$$Gini(D) = 1 - \sum_{i=1}^{k} p_i^2$$where p_i is the probability of an item belonging to class i.Entropy & Information Gain: Measures the amount of impurity or randomness in the data.$$Entropy(D) = -\sum_{i=1}^{k} p_i \log_2(p_i)$$Dataset InformationGet the Palmer Penguins Dataset Extended from Kaggle using the following link:https://www.kaggle.com/datasets/samybaladram/palmers-penguin-dataset-extended The Palmer Penguins dataset contains measurements of penguin species across islands in the Palmer Archipelago:AdelieChinstrapGentooThe primary numerical features are:culmen_length_mm (or bill_length_mm)culmen_depth_mm (or bill_depth_mm)flipper_length_mmbody_mass_gFor visualization, we will use: culmen_length_mm and flipper_length_mm.

Task 1: Import Libraries and Load Dataset

- Load essential libraries (numpy, pandas, matplotlib, seaborn, sklearn).
- Load the Palmer Penguins Extended dataset.

In [ ]:
```
# Step 1: Import essential libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Import Scikit-learn libraries
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, p
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confus

# Step 2: Upload the dataset (Google Colab)
from google.colab import files

print("Please upload the Palmer Penguins Extended CSV file:")
uploaded = files.upload()

# Step 3: Load the uploaded CSV file
import io
```

```
filename = next(
    (name for name in uploaded if name.lower().endswith(".csv")),
    None
)

if filename is None:
    raise ValueError("Please upload a valid CSV file.")

df = pd.read_csv(io.BytesIO(uploaded[filename]))

# Step 4: Display the first five rows
print("\nFirst Five Rows of the Dataset:")
print(df.head())

# Step 5: Display dataset dimensions
print("\nDataset Shape (Rows, Columns):")
print(df.shape)

# Step 6: Display column names
print("\nColumn Names:")
print(df.columns.tolist())

# Step 7: Display dataset information
print("\nDataset Information:")
df.info()

# Step 8: Check for missing values
print("\nMissing Values in Each Column:")
print(df.isnull().sum())

# Step 9: Display statistical summary
print("\nStatistical Summary:")
print(df.describe())

# Step 10: Display categorical column summaries
print("\nCategorical Column Summary:")
print(df.describe(include="object"))

# Step 11: Check duplicate rows
print("\nNumber of Duplicate Rows:")
print(df.duplicated().sum())

# Step 12: Display dataset size
print("\nTotal Number of Records:", len(df))
print("Total Number of Features:", len(df.columns))

print("\nTask 1 completed successfully!")
```

```
Please upload the Palmer Penguins Extended CSV file:
```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving palmerpenguins_extended.csv to palmerpenguins_extended.csv
Saving palmerpenguins_original.csv to palmerpenguins_original.csv

First Five Rows of the Dataset:
  species  island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Biscoe            53.4           17.8              219.0
1  Adelie  Biscoe            49.3           18.1              245.0
2  Adelie  Biscoe            55.7           16.6              226.0
3  Adelie  Biscoe            38.0           15.6              221.0
4  Adelie  Biscoe            60.7           17.9              177.0

   body_mass_g     sex  diet life_stage health_metrics  year
0       5687.0  female  fish      adult     overweight  2021
1       6811.0  female  fish      adult     overweight  2021
2       5388.0  female  fish      adult     overweight  2021
3       6262.0  female  fish      adult     overweight  2021
4       4811.0  female  fish   juvenile     overweight  2021

Dataset Shape (Rows, Columns):
(3430, 11)

Column Names:
['species', 'island', 'bill_length_mm', 'bill_depth_mm', 'flipper_lengt
h_mm', 'body_mass_g', 'sex', 'diet', 'life_stage', 'health_metrics', 'yea
r']

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3430 entries, 0 to 3429
Data columns (total 11 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   species            3430 non-null   object
 1   island             3430 non-null   object
 2   bill_length_mm     3430 non-null   float64
 3   bill_depth_mm      3430 non-null   float64
 4   flipper_length_mm  3430 non-null   float64
 5   body_mass_g        3430 non-null   float64
 6   sex                3430 non-null   object
 7   diet               3430 non-null   object
 8   life_stage         3430 non-null   object
 9   health_metrics     3430 non-null   object
 10  year               3430 non-null   int64
dtypes: float64(4), int64(1), object(6)
memory usage: 294.9+ KB

Missing Values in Each Column:
species              0
island               0
bill_length_mm       0
bill_depth_mm        0
flipper_length_mm    0
body_mass_g          0
sex                  0
diet                 0
```

```
life_stage           0
health_metrics       0
year                 0
dtype: int64

Statistical Summary:
       bill_length_mm  bill_depth_mm  flipper_length_mm   body_mass_g  \
count     3430.000000    3430.000000        3430.000000   3430.000000
mean        38.529825      18.447143         207.028863   4834.710496
std         13.175171       2.774428          28.944765   1311.091310
min         13.600000       9.100000         140.000000   2477.000000
25%         28.900000      16.600000         185.000000   3843.500000
50%         34.500000      18.400000         203.000000   4633.500000
75%         46.600000      20.300000         226.000000   5622.000000
max         88.200000      27.900000         308.000000  10549.000000

              year
count  3430.000000
mean   2023.348397
std       1.313797
min    2021.000000
25%    2022.000000
50%    2024.000000
75%    2024.000000
max    2025.000000

Categorical Column Summary:
       species  island     sex   diet life_stage health_metrics
count     3430    3430    3430   3430       3430           3430
unique       3       3       2      4          3              3
top     Adelie  Biscoe  female  krill   juvenile        healthy
freq      1560    1785    1726   1419       1541           1550

Number of Duplicate Rows:
0

Total Number of Records: 3430
Total Number of Features: 11

Task 1 completed successfully!
```

Task 2: Select Features and Target & Preprocess

- Handle missing values (e.g., drop rows with missing values or impute with median).
- Select features (X) such as culmen_length_mm, culmen_depth_mm, flipper_length_mm, body_mass_g and set species as the target label

(y).

- Encode categorical targets if necessary using LabelEncoder.

In [ ]:

```
# Task 2: Select Features and Target & Preprocess

import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Step 1: Display available column names
print("Available Columns:")
print(df.columns.tolist())

# Step 2: Identify feature columns
# Supports common alternative column names in penguin datasets
possible_features = [
    ["culmen_length_mm", "culmen_depth_mm",
     "flipper_length_mm", "body_mass_g"],
    ["bill_length_mm", "bill_depth_mm",
     "flipper_length_mm", "body_mass_g"]
]

features = next(
    (cols for cols in possible_features
     if all(col in df.columns for col in cols)),
    None
)

if features is None:
    raise ValueError(
        "Required feature columns were not found. "
        "Check the column names printed above."
    )

# Step 3: Identify the target column
target = next(
    (col for col in ["species", "Species"]
     if col in df.columns),
    None
)

if target is None:
    raise ValueError(
        "Species column not found. Check the dataset column names."
    )

# Step 4: Select features (X) and target (y)
data = df[features + [target]].copy()

# Convert feature columns to numeric values
for col in features:
    data[col] = pd.to_numeric(data[col], errors="coerce")

# Step 5: Handle missing values
```

```
print("\nMissing values before preprocessing:")
print(data.isnull().sum())

# Drop rows with missing feature values or target labels
data = data.dropna(subset=features + [target]).copy()

print("\nDataset shape after removing missing values:")
print(data.shape)

# Step 6: Separate features and target
X = data[features]
y = data[target].astype(str).str.strip()

# Remove rows with empty target labels
valid = y.ne("") & y.ne("nan")
X = X.loc[valid].copy()
y = y.loc[valid].copy()

# Step 7: Encode categorical species labels
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

# Step 8: Display the processed data
print("\nSelected Features (X):")
print(X.head())

print("\nOriginal Target Labels (y):")
print(y.head())

print("\nEncoded Target Labels:")
print(y_encoded[:5])

print("\nSpecies Encoding:")
for i, species in enumerate(label_encoder.classes_):
    print(f"{species} -> {i}")

print("\nFinal Feature Shape:", X.shape)
print("Final Target Shape:", y_encoded.shape)

print("\nTask 2 completed successfully!")
```

```
Available Columns:
['species', 'island', 'bill_length_mm', 'bill_depth_mm', 'flipper_lengt
h_mm', 'body_mass_g', 'sex', 'diet', 'life_stage', 'health_metrics', 'yea
r']

Missing values before preprocessing:
bill_length_mm       0
bill_depth_mm        0
flipper_length_mm    0
body_mass_g          0
species              0
dtype: int64

Dataset shape after removing missing values:
(3430, 5)

Selected Features (X):
   bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g
0            53.4           17.8              219.0       5687.0
1            49.3           18.1              245.0       6811.0
2            55.7           16.6              226.0       5388.0
3            38.0           15.6              221.0       6262.0
4            60.7           17.9              177.0       4811.0

Original Target Labels (y):
0    Adelie
1    Adelie
2    Adelie
3    Adelie
4    Adelie
Name: species, dtype: object

Encoded Target Labels:
[0 0 0 0 0]

Species Encoding:
Adelie -> 0
Chinstrap -> 1
Gentoo -> 2

Final Feature Shape: (3430, 4)
Final Target Shape: (3430,)

Task 2 completed successfully!
```

Task 3: Split the Data

- Divide the dataset into training and testing sets using train_test_split.

In [ ]:
```python
# Task 3: Split the Data

from sklearn.model_selection import train_test_split
```

```
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

# Display the shape of the datasets
print("Training and Testing Data Split Successfully!")

print("\nTraining Features Shape:", X_train.shape)
print("Testing Features Shape:", X_test.shape)

print("\nTraining Target Shape:", y_train.shape)
print("Testing Target Shape:", y_test.shape)

# Display the number of training and testing samples
print("\nNumber of Training Samples:", len(X_train))
print("Number of Testing Samples:", len(X_test))

print("\nTask 3 completed successfully!")
```

```
Training and Testing Data Split Successfully!

Training Features Shape: (2744, 4)
Testing Features Shape: (686, 4)

Training Target Shape: (2744,)
Testing Target Shape: (686,)

Number of Training Samples: 2744
Number of Testing Samples: 686

Task 3 completed successfully!
```

Task 4: Implement CART Classifier

- Create a DecisionTreeClassifier model using the Gini criterion (or Entropy).
- Fit the model on the training data and make predictions on the test set.

In [ ]:
```
# Display all actual and predicted species in table format

import pandas as pd

# Convert encoded labels into species names
actual_species = label_encoder.inverse_transform(y_test)
predicted_species = label_encoder.inverse_transform(y_pred)

# Create a DataFrame containing all predictions
```

```python
results = pd.DataFrame({
    "Actual Label": y_test,
    "Predicted Label": y_pred,
    "Actual Species": actual_species,
    "Predicted Species": predicted_species
})

# Add a column indicating whether the prediction is correct
results["Prediction Status"] = (
    results["Actual Species"] == results["Predicted Species"]
).map({True: "Correct", False: "Incorrect"})

# Display all records
print("All Actual vs Predicted Species:")
print(results.to_string(index=False))

# Save all results to a CSV file
results.to_csv("CART_Prediction_Results.csv", index=False)

print("\nTotal Test Records:", len(results))
print("Correct Predictions:", (results["Prediction Status"] == "Correct")
print("Incorrect Predictions:", (results["Prediction Status"] == "Incorre

print("\nAll results saved to CART_Prediction_Results.csv")
```

All Actual vs Predicted Species:

Actual Label	Predicted Label	Actual Species	Predicted Species	Prediction Status
0	2	Adelie	Gentoo	Incorrect
2	2	Gentoo	Gentoo	Correct
0	0	Adelie	Adelie	Correct
0	0	Adelie	Adelie	Correct
0	2	Adelie	Gentoo	Incorrect
2	2	Gentoo	Gentoo	Correct
1	1	Chinstrap	Chinstrap	Correct
0	0	Adelie	Adelie	Correct
2	0	Gentoo	Adelie	Incorrect
0	0	Adelie	Adelie	Correct
0	2	Adelie	Gentoo	Incorrect
0	1	Adelie	Chinstrap	Incorrect
0	0	Adelie	Adelie	Correct
0	0	Adelie	Adelie	Correct
2	1	Gentoo	Chinstrap	Incorrect
2	2	Gentoo	Gentoo	Correct
2	1	Gentoo	Chinstrap	Incorrect
2	0	Gentoo	Adelie	Incorrect
0	0	Adelie	Adelie	Correct
0	1	Adelie	Chinstrap	Incorrect
1	2	Chinstrap	Gentoo	Incorrect
0	1	Adelie	Chinstrap	Incorrect
1	1	Chinstrap	Chinstrap	Correct
1	0	Chinstrap	Adelie	Incorrect
0	0	Adelie	Adelie	Correct
0	0	Adelie	Adelie	Correct

	0	2	Adelie	Gentoo	I
ncorrect					
	1	0	Chinstrap	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	2	0	Gentoo	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	2	0	Gentoo	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	2	1	Gentoo	Chinstrap	I
ncorrect					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	0	0	Adelie	Adelie	
Correct					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	1	0	Chinstrap	Adelie	I
ncorrect					
	2	1	Gentoo	Chinstrap	I
ncorrect					
	0	1	Adelie	Chinstrap	I
ncorrect					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	0	0	Adelie	Adelie	
Correct					
	0	1	Adelie	Chinstrap	I
ncorrect					
	1	1	Chinstrap	Chinstrap	
Correct					
	2	0	Gentoo	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	2	1	Gentoo	Chinstrap	I
ncorrect					
	0	1	Adelie	Chinstrap	I
ncorrect					
	1	0	Chinstrap	Adelie	I

ncorrect	0	2	Adelie	Gentoo	I
ncorrect	1	2	Chinstrap	Gentoo	I
ncorrect	2	0	Gentoo	Adelie	I
ncorrect	2	2	Gentoo	Gentoo	
Correct	1	1	Chinstrap	Chinstrap	
Correct	1	1	Chinstrap	Chinstrap	
Correct	0	0	Adelie	Adelie	
Correct	0	2	Adelie	Gentoo	I
ncorrect	2	1	Gentoo	Chinstrap	I
ncorrect	0	0	Adelie	Adelie	
Correct	0	2	Adelie	Gentoo	I
ncorrect	1	2	Chinstrap	Gentoo	I
ncorrect	0	0	Adelie	Adelie	
Correct	1	1	Chinstrap	Chinstrap	
Correct	0	1	Adelie	Chinstrap	I
ncorrect	0	0	Adelie	Adelie	
Correct	2	2	Gentoo	Gentoo	
Correct	0	0	Adelie	Adelie	
Correct	0	1	Adelie	Chinstrap	I
ncorrect	2	0	Gentoo	Adelie	I
ncorrect	0	0	Adelie	Adelie	
Correct	0	1	Adelie	Chinstrap	I
ncorrect	0	0	Adelie	Adelie	
Correct	2	1	Gentoo	Chinstrap	I
ncorrect	0	0	Adelie	Adelie	
Correct	2	1	Gentoo	Chinstrap	I
ncorrect	2	0	Gentoo	Adelie	I
ncorrect					

	0	0	Adelie	Adelie	
Correct					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	1	2	Chinstrap	Gentoo	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	0	1	Adelie	Chinstrap	I
ncorrect					
	0	2	Adelie	Gentoo	I
ncorrect					
	1	1	Chinstrap	Chinstrap	
Correct					
	1	1	Chinstrap	Chinstrap	
Correct					
	0	1	Adelie	Chinstrap	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	0	1	Adelie	Chinstrap	I
ncorrect					
	2	0	Gentoo	Adelie	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	0	2	Adelie	Gentoo	I
ncorrect					
	1	0	Chinstrap	Adelie	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	1	0	Chinstrap	Adelie	I
ncorrect					
	0	1	Adelie	Chinstrap	I
ncorrect					
	1	0	Chinstrap	Adelie	I
ncorrect					
	1	1	Chinstrap	Chinstrap	
Correct					
	2	0	Gentoo	Adelie	I
ncorrect					
	1	0	Chinstrap	Adelie	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	0	2	Adelie	Gentoo	I
ncorrect					
	2	1	Gentoo	Chinstrap	I

ncorrect				
	2	2	Gentoo	Gentoo
Correct				
	2	2	Gentoo	Gentoo
Correct				
	1	2	Chinstrap	Gentoo
I				
ncorrect				
	0	0	Adelie	Adelie
Correct				
	0	1	Adelie	Chinstrap
I				
ncorrect				
	2	2	Gentoo	Gentoo
Correct				
	2	1	Gentoo	Chinstrap
I				
ncorrect				
	2	0	Gentoo	Adelie
I				
ncorrect				
	0	2	Adelie	Gentoo
I				
ncorrect				
	0	1	Adelie	Chinstrap
I				
ncorrect				
	1	2	Chinstrap	Gentoo
I				
ncorrect				
	2	2	Gentoo	Gentoo
Correct				
	2	1	Gentoo	Chinstrap
I				
ncorrect				
	0	2	Adelie	Gentoo
I				
ncorrect				
	0	1	Adelie	Chinstrap
I				
ncorrect				
	0	0	Adelie	Adelie
Correct				
	0	0	Adelie	Adelie
Correct				
	0	1	Adelie	Chinstrap
I				
ncorrect				
	2	0	Gentoo	Adelie
I				
ncorrect				
	0	0	Adelie	Adelie
Correct				
	1	0	Chinstrap	Adelie
I				
ncorrect				
	1	0	Chinstrap	Adelie
I				
ncorrect				
	0	0	Adelie	Adelie
Correct				
	2	0	Gentoo	Adelie
I				
ncorrect				
	2	1	Gentoo	Chinstrap
I				
ncorrect				
	1	0	Chinstrap	Adelie
I				
ncorrect				
	0	2	Adelie	Gentoo
I				
ncorrect				

0	0	Adelie	Adelie	Correct
2	0	Gentoo	Adelie	Incorrect
0	1	Adelie	Chinstrap	Incorrect
0	1	Adelie	Chinstrap	Incorrect
2	2	Gentoo	Gentoo	Correct
0	1	Adelie	Chinstrap	Incorrect
2	0	Gentoo	Adelie	Incorrect
1	0	Chinstrap	Adelie	Incorrect
0	2	Adelie	Gentoo	Incorrect
0	0	Adelie	Adelie	Correct
2	0	Gentoo	Adelie	Incorrect
2	2	Gentoo	Gentoo	Correct
0	2	Adelie	Gentoo	Incorrect
1	2	Chinstrap	Gentoo	Incorrect
0	0	Adelie	Adelie	Correct
2	2	Gentoo	Gentoo	Correct
2	0	Gentoo	Adelie	Incorrect
0	0	Adelie	Adelie	Correct
0	0	Adelie	Adelie	Correct
0	2	Adelie	Gentoo	Incorrect
2	2	Gentoo	Gentoo	Correct
0	1	Adelie	Chinstrap	Incorrect
1	2	Chinstrap	Gentoo	Incorrect
1	1	Chinstrap	Chinstrap	Correct
1	0	Chinstrap	Adelie	Incorrect
0	0	Adelie	Adelie	Correct
2	1	Gentoo	Chinstrap	Incorrect
0	0	Adelie	Adelie	

Correct
2 0 Gentoo Adelie I
ncorrect
0 0 Adelie Adelie
Correct
2 0 Gentoo Adelie I
ncorrect
0 1 Adelie Chinstrap I
ncorrect
0 0 Adelie Adelie
Correct
1 2 Chinstrap Gentoo I
ncorrect
1 2 Chinstrap Gentoo I
ncorrect
1 0 Chinstrap Adelie I
ncorrect
0 1 Adelie Chinstrap I
ncorrect
1 1 Chinstrap Chinstrap
Correct
0 2 Adelie Gentoo I
ncorrect
0 1 Adelie Chinstrap I
ncorrect
2 1 Gentoo Chinstrap I
ncorrect
0 0 Adelie Adelie
Correct
2 0 Gentoo Adelie I
ncorrect
0 2 Adelie Gentoo I
ncorrect
2 2 Gentoo Gentoo
Correct
0 0 Adelie Adelie
Correct
2 0 Gentoo Adelie I
ncorrect
0 2 Adelie Gentoo I
ncorrect
0 0 Adelie Adelie
Correct
1 2 Chinstrap Gentoo I
ncorrect
1 1 Chinstrap Chinstrap
Correct
0 0 Adelie Adelie
Correct
0 2 Adelie Gentoo I
ncorrect
0 2 Adelie Gentoo I
ncorrect
1 2 Chinstrap Gentoo I
ncorrect

	1	0	Chinstrap	Adelie	I
ncorrect					
	1	1	Chinstrap	Chinstrap	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	0	0	Adelie	Adelie	
Correct					
	0	0	Adelie	Adelie	
Correct					
	0	1	Adelie	Chinstrap	I
ncorrect					
	1	0	Chinstrap	Adelie	I
ncorrect					
	1	2	Chinstrap	Gentoo	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	0	1	Adelie	Chinstrap	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	0	2	Adelie	Gentoo	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	0	0	Adelie	Adelie	
Correct					
	2	0	Gentoo	Adelie	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	2	1	Gentoo	Chinstrap	I
ncorrect					
	0	2	Adelie	Gentoo	I
ncorrect					
	2	0	Gentoo	Adelie	I
ncorrect					
	0	2	Adelie	Gentoo	I
ncorrect					
	2	0	Gentoo	Adelie	I
ncorrect					
	1	0	Chinstrap	Adelie	I
ncorrect					
	1	2	Chinstrap	Gentoo	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	1	0	Chinstrap	Adelie	I
ncorrect					
	1	2	Chinstrap	Gentoo	I
ncorrect					
	2	0	Gentoo	Adelie	I

ncorrect					
	2	0	Gentoo	Adelie	I
ncorrect					
	1	1	Chinstrap	Chinstrap	
Correct					
	0	0	Adelie	Adelie	
Correct					
	2	0	Gentoo	Adelie	I
ncorrect					
	2	0	Gentoo	Adelie	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	2	0	Gentoo	Adelie	I
ncorrect					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	0	0	Adelie	Adelie	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	1	2	Chinstrap	Gentoo	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	0	1	Adelie	Chinstrap	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	0	0	Adelie	Adelie	
Correct					
	1	0	Chinstrap	Adelie	I
ncorrect					
	2	1	Gentoo	Chinstrap	I
ncorrect					
	0	1	Adelie	Chinstrap	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	0	2	Adelie	Gentoo	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	2	0	Gentoo	Adelie	I
ncorrect					

	2	1	Gentoo	Chinstrap	Incorrect
	0	0	Adelie	Adelie	Correct
	2	2	Gentoo	Gentoo	Correct
	0	1	Adelie	Chinstrap	Incorrect
	1	2	Chinstrap	Gentoo	Incorrect
	1	1	Chinstrap	Chinstrap	Correct
	0	1	Adelie	Chinstrap	Incorrect
	2	1	Gentoo	Chinstrap	Incorrect
	0	0	Adelie	Adelie	Correct
	2	0	Gentoo	Adelie	Incorrect
	0	1	Adelie	Chinstrap	Incorrect
	0	0	Adelie	Adelie	Correct
	0	2	Adelie	Gentoo	Incorrect
	0	0	Adelie	Adelie	Correct
	2	2	Gentoo	Gentoo	Correct
	0	0	Adelie	Adelie	Correct
	2	2	Gentoo	Gentoo	Correct
	2	1	Gentoo	Chinstrap	Incorrect
	2	1	Gentoo	Chinstrap	Incorrect
	0	0	Adelie	Adelie	Correct
	1	0	Chinstrap	Adelie	Incorrect
	0	0	Adelie	Adelie	Correct
	1	1	Chinstrap	Chinstrap	Correct
	2	2	Gentoo	Gentoo	Correct
	2	2	Gentoo	Gentoo	Correct
	2	0	Gentoo	Adelie	Incorrect
	1	0	Chinstrap	Adelie	Incorrect
	2	2	Gentoo	Gentoo	

Correct
0 0 Adelie Adelie
Correct
0 1 Adelie Chinstrap I
ncorrect
2 0 Gentoo Adelie I
ncorrect
2 2 Gentoo Gentoo
Correct
0 1 Adelie Chinstrap I
ncorrect
0 0 Adelie Adelie
Correct
2 0 Gentoo Adelie I
ncorrect
2 0 Gentoo Adelie I
ncorrect
1 2 Chinstrap Gentoo I
ncorrect
2 0 Gentoo Adelie I
ncorrect
2 2 Gentoo Gentoo
Correct
1 2 Chinstrap Gentoo I
ncorrect
2 2 Gentoo Gentoo
Correct
1 2 Chinstrap Gentoo I
ncorrect
0 2 Adelie Gentoo I
ncorrect
0 0 Adelie Adelie
Correct
2 2 Gentoo Gentoo
Correct
1 1 Chinstrap Chinstrap
Correct
0 2 Adelie Gentoo I
ncorrect
0 0 Adelie Adelie
Correct
2 1 Gentoo Chinstrap I
ncorrect
0 2 Adelie Gentoo I
ncorrect
2 1 Gentoo Chinstrap I
ncorrect
1 2 Chinstrap Gentoo I
ncorrect
2 2 Gentoo Gentoo
Correct
2 0 Gentoo Adelie I
ncorrect
0 2 Adelie Gentoo I
ncorrect

2	0	Gentoo	Adelie	Incorrect
0	0	Adelie	Adelie	Correct
2	2	Gentoo	Gentoo	Correct
2	2	Gentoo	Gentoo	Correct
2	2	Gentoo	Gentoo	Correct
0	0	Adelie	Adelie	Correct
0	0	Adelie	Adelie	Correct
0	1	Adelie	Chinstrap	Incorrect
2	1	Gentoo	Chinstrap	Incorrect
0	0	Adelie	Adelie	Correct
0	1	Adelie	Chinstrap	Incorrect
2	1	Gentoo	Chinstrap	Incorrect
2	1	Gentoo	Chinstrap	Incorrect
0	0	Adelie	Adelie	Correct
0	1	Adelie	Chinstrap	Incorrect
0	0	Adelie	Adelie	Correct
1	2	Chinstrap	Gentoo	Incorrect
2	2	Gentoo	Gentoo	Correct
2	2	Gentoo	Gentoo	Correct
1	0	Chinstrap	Adelie	Incorrect
2	1	Gentoo	Chinstrap	Incorrect
2	2	Gentoo	Gentoo	Correct
2	2	Gentoo	Gentoo	Correct
1	0	Chinstrap	Adelie	Incorrect
1	1	Chinstrap	Chinstrap	Correct
1	1	Chinstrap	Chinstrap	Correct
0	0	Adelie	Adelie	Correct
0	2	Adelie	Gentoo	I

ncorrect				
	0	0	Adelie	Adelie
Correct				
	1	1	Chinstrap	Chinstrap
Correct				
	0	2	Adelie	Gentoo
I				
ncorrect				
	2	0	Gentoo	Adelie
I				
ncorrect				
	0	2	Adelie	Gentoo
I				
ncorrect				
	1	1	Chinstrap	Chinstrap
Correct				
	0	0	Adelie	Adelie
Correct				
	2	2	Gentoo	Gentoo
Correct				
	0	2	Adelie	Gentoo
I				
ncorrect				
	0	0	Adelie	Adelie
Correct				
	2	1	Gentoo	Chinstrap
I				
ncorrect				
	0	1	Adelie	Chinstrap
I				
ncorrect				
	0	0	Adelie	Adelie
Correct				
	0	1	Adelie	Chinstrap
I				
ncorrect				
	2	2	Gentoo	Gentoo
Correct				
	0	0	Adelie	Adelie
Correct				
	0	1	Adelie	Chinstrap
I				
ncorrect				
	1	2	Chinstrap	Gentoo
I				
ncorrect				
	0	0	Adelie	Adelie
Correct				
	0	2	Adelie	Gentoo
I				
ncorrect				
	2	0	Gentoo	Adelie
I				
ncorrect				
	0	2	Adelie	Gentoo
I				
ncorrect				
	0	2	Adelie	Gentoo
I				
ncorrect				
	0	2	Adelie	Gentoo
I				
ncorrect				
	0	2	Adelie	Gentoo
I				
ncorrect				
	1	1	Chinstrap	Chinstrap
Correct				
	1	2	Chinstrap	Gentoo
I				
ncorrect				

2	2	Gentoo	Gentoo	Correct
0	1	Adelie	Chinstrap	Incorrect
0	2	Adelie	Gentoo	Incorrect
2	1	Gentoo	Chinstrap	Incorrect
2	2	Gentoo	Gentoo	Correct
0	0	Adelie	Adelie	Correct
2	2	Gentoo	Gentoo	Correct
0	1	Adelie	Chinstrap	Incorrect
0	1	Adelie	Chinstrap	Incorrect
0	0	Adelie	Adelie	Correct
1	2	Chinstrap	Gentoo	Incorrect
2	0	Gentoo	Adelie	Incorrect
2	2	Gentoo	Gentoo	Correct
0	1	Adelie	Chinstrap	Incorrect
2	2	Gentoo	Gentoo	Correct
1	0	Chinstrap	Adelie	Incorrect
1	1	Chinstrap	Chinstrap	Correct
0	2	Adelie	Gentoo	Incorrect
0	1	Adelie	Chinstrap	Incorrect
0	2	Adelie	Gentoo	Incorrect
0	0	Adelie	Adelie	Correct
0	1	Adelie	Chinstrap	Incorrect
0	2	Adelie	Gentoo	Incorrect
0	2	Adelie	Gentoo	Incorrect
2	2	Gentoo	Gentoo	Correct
1	0	Chinstrap	Adelie	Incorrect
1	1	Chinstrap	Chinstrap	Correct
1	1	Chinstrap	Chinstrap	

0	0	Adelie	Adelie	Correct
1	2	Chinstrap	Gentoo	Incorrect
1	2	Chinstrap	Gentoo	Incorrect
0	0	Adelie	Adelie	Correct
2	0	Gentoo	Adelie	Incorrect
0	1	Adelie	Chinstrap	Incorrect
2	2	Gentoo	Gentoo	Correct
2	1	Gentoo	Chinstrap	Incorrect
2	2	Gentoo	Gentoo	Correct
0	2	Adelie	Gentoo	Incorrect
0	0	Adelie	Adelie	Correct
0	2	Adelie	Gentoo	Incorrect
2	0	Gentoo	Adelie	Incorrect
0	1	Adelie	Chinstrap	Incorrect
2	1	Gentoo	Chinstrap	Incorrect
2	1	Gentoo	Chinstrap	Incorrect
0	0	Adelie	Adelie	Correct
0	2	Adelie	Gentoo	Incorrect
2	1	Gentoo	Chinstrap	Incorrect
0	0	Adelie	Adelie	Correct
2	2	Gentoo	Gentoo	Correct
0	2	Adelie	Gentoo	Incorrect
2	2	Gentoo	Gentoo	Correct
2	2	Gentoo	Gentoo	Correct
0	2	Adelie	Gentoo	Incorrect
2	0	Gentoo	Adelie	Incorrect
1	0	Chinstrap	Adelie	Incorrect

0	2	Adelie	Gentoo	I
ncorrect				
1	2	Chinstrap	Gentoo	I
ncorrect				
0	2	Adelie	Gentoo	I
ncorrect				
2	0	Gentoo	Adelie	I
ncorrect				
2	1	Gentoo	Chinstrap	I
ncorrect				
1	2	Chinstrap	Gentoo	I
ncorrect				
0	0	Adelie	Adelie	
Correct				
2	1	Gentoo	Chinstrap	I
ncorrect				
0	0	Adelie	Adelie	
Correct				
2	0	Gentoo	Adelie	I
ncorrect				
1	1	Chinstrap	Chinstrap	
Correct				
0	0	Adelie	Adelie	
Correct				
2	2	Gentoo	Gentoo	
Correct				
0	0	Adelie	Adelie	
Correct				
2	2	Gentoo	Gentoo	
Correct				
2	2	Gentoo	Gentoo	
Correct				
2	0	Gentoo	Adelie	I
ncorrect				
1	2	Chinstrap	Gentoo	I
ncorrect				
2	2	Gentoo	Gentoo	
Correct				
0	2	Adelie	Gentoo	I
ncorrect				
2	1	Gentoo	Chinstrap	I
ncorrect				
1	2	Chinstrap	Gentoo	I
ncorrect				
2	0	Gentoo	Adelie	I
ncorrect				
2	1	Gentoo	Chinstrap	I
ncorrect				
2	1	Gentoo	Chinstrap	I
ncorrect				
0	1	Adelie	Chinstrap	I
ncorrect				
2	0	Gentoo	Adelie	I
ncorrect				
0	0	Adelie	Adelie	

Correct
0 1 Adelie Chinstrap I
ncorrect
2 2 Gentoo Gentoo
Correct
0 0 Adelie Adelie
Correct
0 1 Adelie Chinstrap I
ncorrect
2 2 Gentoo Gentoo
Correct
2 2 Gentoo Gentoo
Correct
0 0 Adelie Adelie
Correct
0 2 Adelie Gentoo I
ncorrect
1 2 Chinstrap Gentoo I
ncorrect
0 0 Adelie Adelie
Correct
1 0 Chinstrap Adelie I
ncorrect
1 0 Chinstrap Adelie I
ncorrect
0 0 Adelie Adelie
Correct
0 0 Adelie Adelie
Correct
2 1 Gentoo Chinstrap I
ncorrect
1 1 Chinstrap Chinstrap
Correct
0 0 Adelie Adelie
Correct
2 2 Gentoo Gentoo
Correct
2 2 Gentoo Gentoo
Correct
2 2 Gentoo Gentoo
Correct
0 0 Adelie Adelie
Correct
2 0 Gentoo Adelie I
ncorrect
0 1 Adelie Chinstrap I
ncorrect
2 0 Gentoo Adelie I
ncorrect
2 2 Gentoo Gentoo
Correct
0 2 Adelie Gentoo I
ncorrect
2 2 Gentoo Gentoo
Correct

2	0	Gentoo	Adelie	Incorrect
2	0	Gentoo	Adelie	Incorrect
0	2	Adelie	Gentoo	Incorrect
0	0	Adelie	Adelie	Correct
1	1	Chinstrap	Chinstrap	Correct
1	0	Chinstrap	Adelie	Incorrect
2	1	Gentoo	Chinstrap	Incorrect
0	2	Adelie	Gentoo	Incorrect
2	0	Gentoo	Adelie	Incorrect
2	2	Gentoo	Gentoo	Correct
0	0	Adelie	Adelie	Correct
0	2	Adelie	Gentoo	Incorrect
0	0	Adelie	Adelie	Correct
2	0	Gentoo	Adelie	Incorrect
0	1	Adelie	Chinstrap	Incorrect
2	2	Gentoo	Gentoo	Correct
2	1	Gentoo	Chinstrap	Incorrect
0	0	Adelie	Adelie	Correct
0	0	Adelie	Adelie	Correct
2	0	Gentoo	Adelie	Incorrect
0	2	Adelie	Gentoo	Incorrect
2	2	Gentoo	Gentoo	Correct
2	2	Gentoo	Gentoo	Correct
0	2	Adelie	Gentoo	Incorrect
0	2	Adelie	Gentoo	Incorrect
2	2	Gentoo	Gentoo	Correct
2	0	Gentoo	Adelie	Incorrect
2	1	Gentoo	Chinstrap	I

ncorrect					
	2	0	Gentoo	Adelie	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	2	0	Gentoo	Adelie	I
ncorrect					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	1	0	Chinstrap	Adelie	I
ncorrect					
	0	2	Adelie	Gentoo	I
ncorrect					
	1	2	Chinstrap	Gentoo	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	2	0	Gentoo	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	0	0	Adelie	Adelie	
Correct					
	0	2	Adelie	Gentoo	I
ncorrect					
	1	0	Chinstrap	Adelie	I
ncorrect					
	1	2	Chinstrap	Gentoo	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	1	0	Chinstrap	Adelie	I
ncorrect					
	0	1	Adelie	Chinstrap	I
ncorrect					
	1	2	Chinstrap	Gentoo	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	0	0	Adelie	Adelie	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	1	1	Chinstrap	Chinstrap	
Correct					
	2	2	Gentoo	Gentoo	
Correct					

0	0	Adelie	Adelie	Correct
2	2	Gentoo	Gentoo	Correct
2	2	Gentoo	Gentoo	Correct
2	2	Gentoo	Gentoo	Correct
2	0	Gentoo	Adelie	Incorrect
0	2	Adelie	Gentoo	Incorrect
0	0	Adelie	Adelie	Correct
2	1	Gentoo	Chinstrap	Incorrect
0	0	Adelie	Adelie	Correct
0	0	Adelie	Adelie	Correct
2	2	Gentoo	Gentoo	Correct
1	2	Chinstrap	Gentoo	Incorrect
0	2	Adelie	Gentoo	Incorrect
0	1	Adelie	Chinstrap	Incorrect
1	0	Chinstrap	Adelie	Incorrect
2	2	Gentoo	Gentoo	Correct
1	2	Chinstrap	Gentoo	Incorrect
0	0	Adelie	Adelie	Correct
1	0	Chinstrap	Adelie	Incorrect
1	0	Chinstrap	Adelie	Incorrect
0	0	Adelie	Adelie	Correct
2	0	Gentoo	Adelie	Incorrect
2	2	Gentoo	Gentoo	Correct
0	2	Adelie	Gentoo	Incorrect
0	0	Adelie	Adelie	Correct
1	2	Chinstrap	Gentoo	Incorrect
1	0	Chinstrap	Adelie	Incorrect
0	1	Adelie	Chinstrap	Incorrect

ncorrect					
	2	0	Gentoo	Adelie	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	0	2	Adelie	Gentoo	I
ncorrect					
	1	2	Chinstrap	Gentoo	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	2	1	Gentoo	Chinstrap	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	0	0	Adelie	Adelie	
Correct					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	1	1	Chinstrap	Chinstrap	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	2	1	Gentoo	Chinstrap	I
ncorrect					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	1	Adelie	Chinstrap	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	1	0	Chinstrap	Adelie	I
ncorrect					
	1	0	Chinstrap	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	1	1	Chinstrap	Chinstrap	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	2	0	Gentoo	Adelie	I
ncorrect					

	0	0	Adelie	Adelie	
Correct					
	2	0	Gentoo	Adelie	I
ncorrect					
	1	0	Chinstrap	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	0	0	Adelie	Adelie	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	1	2	Chinstrap	Gentoo	I
ncorrect					
	1	0	Chinstrap	Adelie	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	2	0	Gentoo	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	2	1	Gentoo	Chinstrap	I
ncorrect					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	1	Adelie	Chinstrap	I
ncorrect					
	2	0	Gentoo	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	2	Adelie	Gentoo	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	0	0	Adelie	Adelie	
Correct					
	1	2	Chinstrap	Gentoo	I
ncorrect					
	1	0	Chinstrap	Adelie	I
ncorrect					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	2	Adelie	Gentoo	I

ncorrect					
	0	0	Adelie	Adelie	
Correct					
	0	0	Adelie	Adelie	
Correct					
	1	0	Chinstrap	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	0	1	Adelie	Chinstrap	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	1	0	Chinstrap	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	0	1	Adelie	Chinstrap	I
ncorrect					
	2	0	Gentoo	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	2	0	Gentoo	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	0	1	Adelie	Chinstrap	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	2	0	Gentoo	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	1	0	Chinstrap	Adelie	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	1	1	Chinstrap	Chinstrap	
Correct					
	0	1	Adelie	Chinstrap	I
ncorrect					
	2	2	Gentoo	Gentoo	
Correct					
	0	0	Adelie	Adelie	
Correct					
	0	0	Adelie	Adelie	
Correct					
	0	0	Adelie	Adelie	
Correct					

0	0	Adelie	Adelie	Correct
2	0	Gentoo	Adelie	Incorrect
2	1	Gentoo	Chinstrap	Incorrect
2	2	Gentoo	Gentoo	Correct
2	2	Gentoo	Gentoo	Correct
0	2	Adelie	Gentoo	Incorrect
0	0	Adelie	Adelie	Correct
0	0	Adelie	Adelie	Correct
0	1	Adelie	Chinstrap	Incorrect
2	2	Gentoo	Gentoo	Correct
2	0	Gentoo	Adelie	Incorrect
2	2	Gentoo	Gentoo	Correct
2	2	Gentoo	Gentoo	Correct
2	1	Gentoo	Chinstrap	Incorrect
0	1	Adelie	Chinstrap	Incorrect
0	2	Adelie	Gentoo	Incorrect
2	0	Gentoo	Adelie	Incorrect
0	2	Adelie	Gentoo	Incorrect
2	1	Gentoo	Chinstrap	Incorrect
2	0	Gentoo	Adelie	Incorrect
2	2	Gentoo	Gentoo	Correct
2	2	Gentoo	Gentoo	Correct
2	2	Gentoo	Gentoo	Correct
0	2	Adelie	Gentoo	Incorrect
2	2	Gentoo	Gentoo	Correct
2	1	Gentoo	Chinstrap	Incorrect
2	2	Gentoo	Gentoo	Correct
0	0	Adelie	Adelie	

Correct					
	0	2	Adelie	Gentoo	I
ncorrect					
	1	2	Chinstrap	Gentoo	I
ncorrect					
	1	0	Chinstrap	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	1	1	Chinstrap	Chinstrap	
Correct					
	0	2	Adelie	Gentoo	I
ncorrect					
	1	2	Chinstrap	Gentoo	I
ncorrect					
	1	0	Chinstrap	Adelie	I
ncorrect					
	2	0	Gentoo	Adelie	I
ncorrect					
	1	0	Chinstrap	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	0	0	Adelie	Adelie	
Correct					
	2	2	Gentoo	Gentoo	
Correct					
	1	0	Chinstrap	Adelie	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	2	1	Gentoo	Chinstrap	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	2	0	Gentoo	Adelie	I
ncorrect					
	0	2	Adelie	Gentoo	I
ncorrect					
	2	1	Gentoo	Chinstrap	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					
	0	1	Adelie	Chinstrap	I
ncorrect					
	1	2	Chinstrap	Gentoo	I
ncorrect					
	0	2	Adelie	Gentoo	I
ncorrect					
	0	0	Adelie	Adelie	
Correct					

```
Total Test Records: 686
Correct Predictions: 310
Incorrect Predictions: 376

All results saved to CART_Prediction_Results.csv
```

Task 5: Evaluate CART Model

- Accuracy: Calculate classification accuracy.
- Confusion Matrix: Generate the matrix to analyze misclassifications across species.
- Classification Report: Output Precision, Recall, F1-Score, and Support.

In [ ]:
```python
# Task 5: Evaluate CART Model

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Step 1: Calculate classification accuracy
accuracy = accuracy_score(y_test, y_pred)

print("CART Model Evaluation")
print("----------------------")
print(f"Classification Accuracy: {accuracy * 100:.2f}%")

# Step 2: Generate confusion matrix
cm = confusion_matrix(
    y_test,
    y_pred,
    labels=range(len(label_encoder.classes_))
)

print("\nConfusion Matrix:")
print(cm)

# Visualize the confusion matrix
plt.figure(figsize=(7, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=label_encoder.classes_,
    yticklabels=label_encoder.classes_
```

```
)

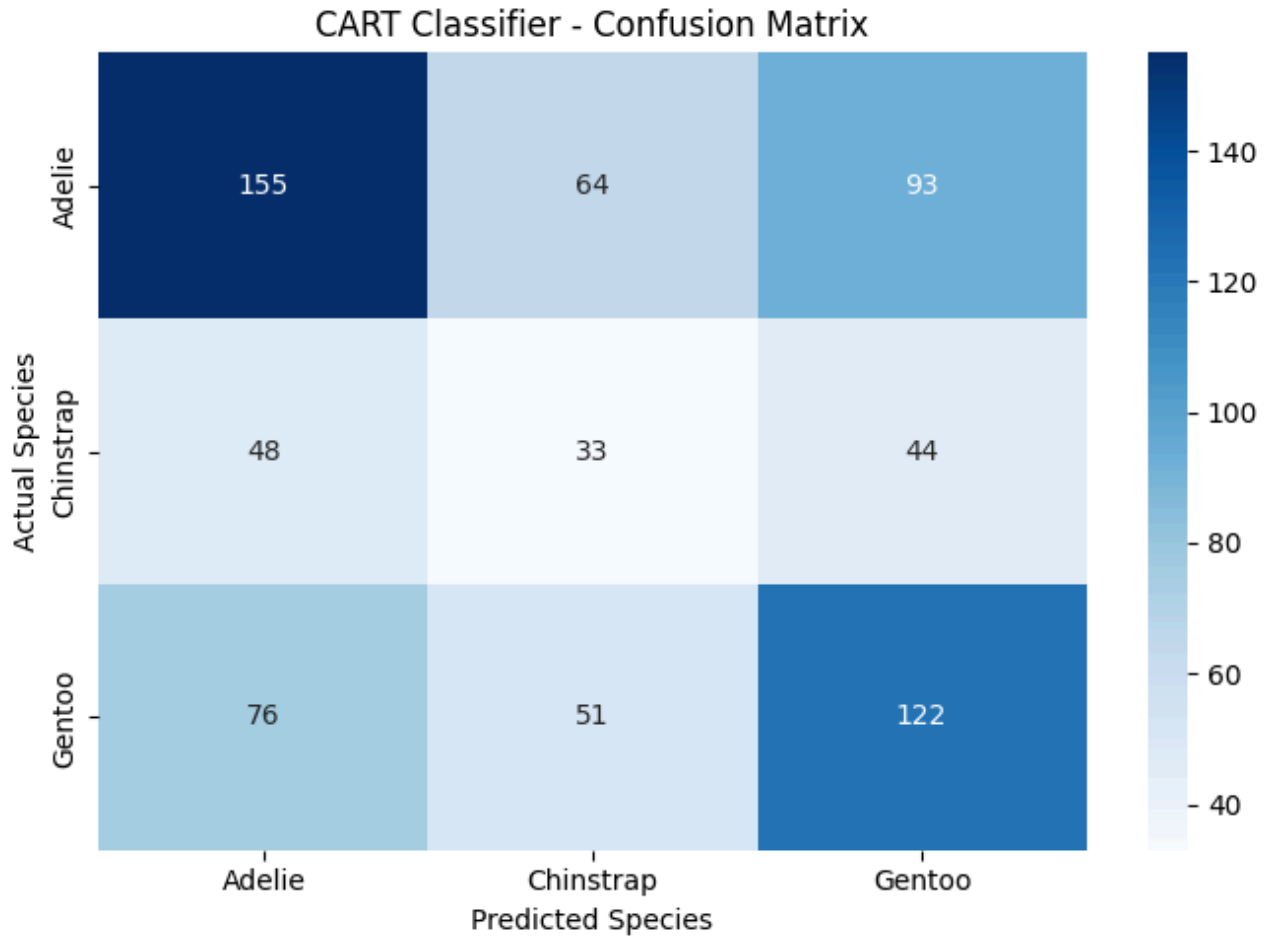
plt.title("CART Classifier - Confusion Matrix")
plt.xlabel("Predicted Species")
plt.ylabel("Actual Species")
plt.tight_layout()
plt.show()

# Step 3: Generate classification report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        labels=range(len(label_encoder.classes_)),
        target_names=label_encoder.classes_,
        zero_division=0
    )
)

print("\nTask 5 completed successfully!")
```

```
CART Model Evaluation
---------------------
Classification Accuracy: 45.19%

Confusion Matrix:
[[155  64  93]
 [ 48  33  44]
 [ 76  51 122]]
```




```
Classification Report:
              precision    recall  f1-score   support

      Adelie       0.56      0.50      0.52       312
   Chinstrap       0.22      0.26      0.24       125
      Gentoo       0.47      0.49      0.48       249

    accuracy                           0.45       686
   macro avg       0.42      0.42      0.42       686
weighted avg       0.46      0.45      0.46       686


Task 5 completed successfully!
```

Task 6: Visualize Decision Tree & Decision Boundary

- Visualize Tree Structure: Plot the decision tree diagram using sklearn.tree.plot_tree.
- Select 2-D Features: Select culmen_length_mm and flipper_length_mm.

- Train 2-D Model & Plot Boundary: Train the decision tree on 2D data and plot axis-aligned rectangular decision boundaries along with data points

In [ ]:

```python
# TASK 6: VISUALIZE DECISION TREE & DECISION BOUNDARY

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score

# 1. Load dataset
df = sns.load_dataset("penguins")

df = df.rename(columns={
    "bill_length_mm": "culmen_length_mm"
})

# 2. Select features and target
features = ["culmen_length_mm", "flipper_length_mm"]
target = "species"

df = df[features + [target]].dropna(subset=[target])

X = df[features]
y = df[target]

# Fill missing feature values
imputer = SimpleImputer(strategy="median")
X = pd.DataFrame(
    imputer.fit_transform(X),
    columns=features,
    index=df.index
)

# 3. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 4. Train decision tree
model = DecisionTreeClassifier(
    max_depth=3,
    min_samples_leaf=5,
    random_state=42
)
```

```python
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Decision Tree Accuracy:",
      round(accuracy_score(y_test, y_pred) * 100, 2), "%")

# ================================================
# OUTPUT 1: DECISION TREE
# ================================================

plt.figure(figsize=(20, 10))

plot_tree(
    model,
    feature_names=features,
    class_names=[str(c) for c in model.classes_],
    filled=True,
    rounded=True,
    impurity=False,
    fontsize=10,
    precision=2
)

plt.title("Decision Tree Classification", fontsize=18)
plt.tight_layout()
plt.show()

# ================================================
# OUTPUT 2: 2D DECISION BOUNDARY
# ================================================

# Train model using the two selected features
dt = DecisionTreeClassifier(
    max_depth=4,
    min_samples_leaf=5,
    random_state=42
)

dt.fit(X_train, y_train)

# Create mesh grid
x_min = X[features[0]].min() - 3
x_max = X[features[0]].max() + 3
y_min = X[features[1]].min() - 10
y_max = X[features[1]].max() + 10

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 400),
    np.linspace(y_min, y_max, 400)
)

grid = pd.DataFrame({
```

```
    features[0]: xx.ravel(),
    features[1]: yy.ravel()
})

# Predict grid points
predictions = dt.predict(grid)

class_names = list(dt.classes_)
class_map = {name: i for i, name in enumerate(class_names)}

Z = np.array([
    class_map[name] for name in predictions
]).reshape(xx.shape)

# Plot decision regions
plt.figure(figsize=(12, 8))

plt.contourf(
    xx, yy, Z,
    levels=np.arange(len(class_names) + 1) - 0.5,
    cmap=plt.cm.Set2,
    alpha=0.5
)

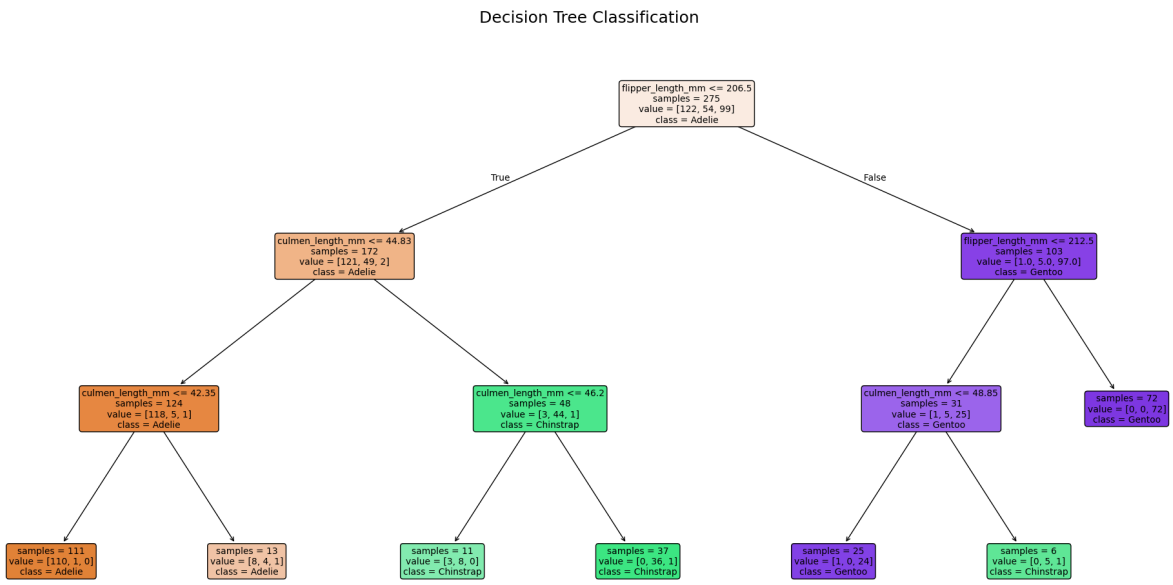
# Plot decision boundaries
plt.contour(
    xx, yy, Z,
    levels=np.arange(len(class_names) - 1) + 0.5,
    colors="black",
    linewidths=1.5
)

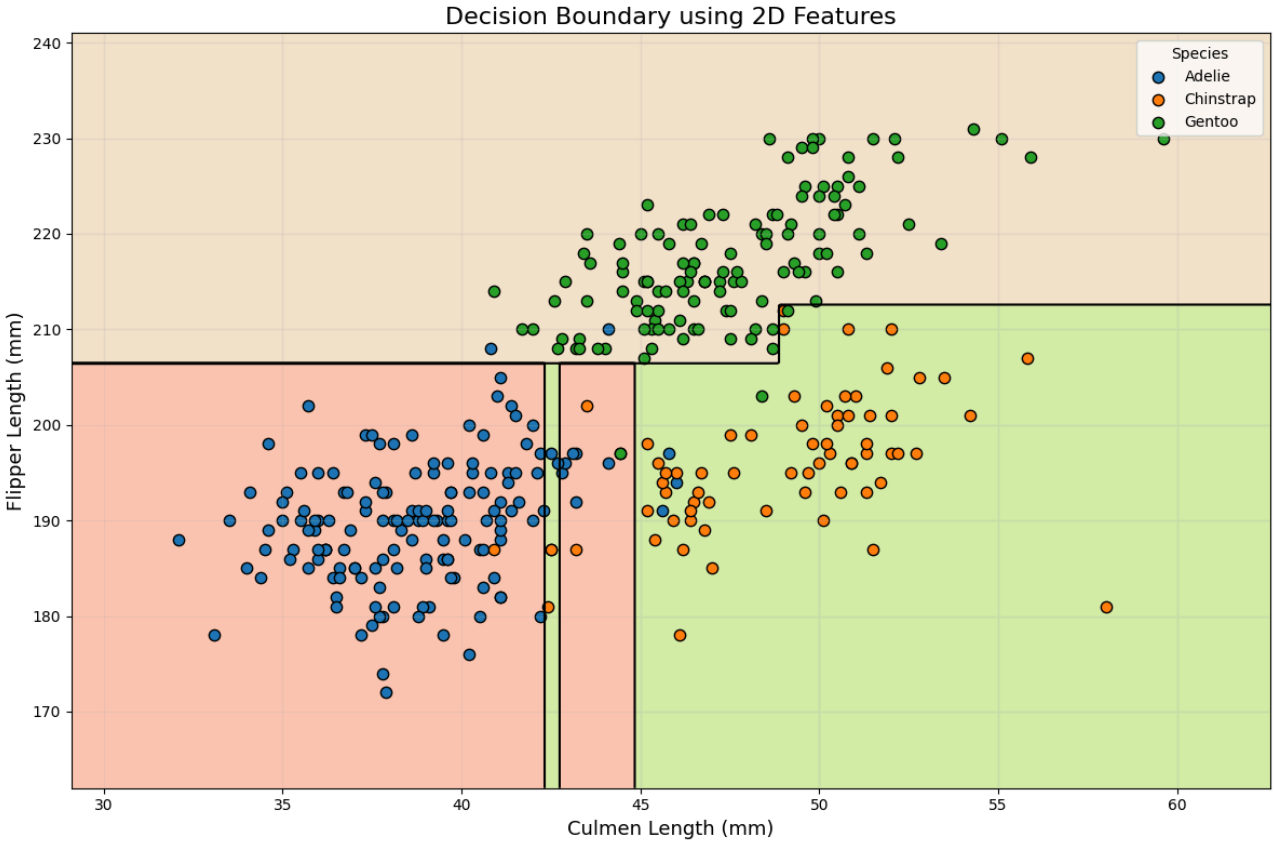
# Plot actual data points
for species in class_names:
    mask = y == species

    plt.scatter(
        X.loc[mask, "culmen_length_mm"],
        X.loc[mask, "flipper_length_mm"],
        label=species,
        edgecolors="black",
        s=55
    )

plt.xlabel("Culmen Length (mm)", fontsize=13)
plt.ylabel("Flipper Length (mm)", fontsize=13)
plt.title("Decision Boundary using 2D Features", fontsize=16)
plt.legend(title="Species")
plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()
```

```
Decision Tree Accuracy: 97.1 %
```





Task 7: Normal Linear Classification Boundary

- Train a Logistic Regression model using the same two features as a normal

linear baseline.

- Visualize its linear decision boundary.

In [ ]:

```
# ============================================================
# TASK 7: NORMAL LINEAR CLASSIFICATION BOUNDARY
# Logistic Regression using 2D features
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# 1. Load the Palmer Penguins dataset
df = sns.load_dataset("penguins")

# Rename feature to match the assignment
df = df.rename(columns={
    "bill_length_mm": "culmen_length_mm"
})

# 2. Select the same two features as Task 6
features = ["culmen_length_mm", "flipper_length_mm"]
target = "species"

data = df[features + [target]].dropna(subset=[target]).copy()

X = data[features]
y = data[target]

# 3. Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# 4. Build Logistic Regression model
# Imputation and scaling are fitted using training data only
model = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler(),
    LogisticRegression(max_iter=2000)
)
```

```
model.fit(X_train, y_train)

# 5. Evaluate the model
y_pred = model.predict(X_test)

print("========== LOGISTIC REGRESSION ==========")
print("Accuracy: {:.2f}%".format(
    accuracy_score(y_test, y_pred) * 100
))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# 6. Create a grid for the decision boundary
x_min = X["culmen_length_mm"].min() - 3
x_max = X["culmen_length_mm"].max() + 3

y_min = X["flipper_length_mm"].min() - 10
y_max = X["flipper_length_mm"].max() + 10

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 500),
    np.linspace(y_min, y_max, 500)
)

grid = pd.DataFrame({
    "culmen_length_mm": xx.ravel(),
    "flipper_length_mm": yy.ravel()
})

# Predict the species at every grid point
Z = model.predict(grid)

class_names = list(model.classes_)
class_map = {name: i for i, name in enumerate(class_names)}

Z = np.array([class_map[name] for name in Z]).reshape(xx.shape)

# 7. Plot the linear decision regions
fig, ax = plt.subplots(figsize=(12, 8), dpi=150)

colors = ["#A8DADC", "#FFD6A5", "#CDB4DB"]

ax.contourf(
    xx,
    yy,
    Z,
    levels=np.arange(len(class_names) + 1) - 0.5,
    colors=colors[:len(class_names)],
    alpha=0.55
)

# Plot the boundaries between predicted classes
ax.contour(
```

```
    xx,
    yy,
    Z,
    levels=np.arange(len(class_names) - 1) + 0.5,
    colors="black",
    linewidths=1.5
)

# 8. Plot actual data points
for i, species in enumerate(class_names):
    mask = y == species

    ax.scatter(
        X.loc[mask, "culmen_length_mm"],
        X.loc[mask, "flipper_length_mm"],
        color=colors[i],
        edgecolors="black",
        s=55,
        label=species,
        alpha=0.9
    )

# 9. Add labels and title
ax.set_xlabel("Culmen Length (mm)", fontsize=13)
ax.set_ylabel("Flipper Length (mm)", fontsize=13)

ax.set_title(
    "Logistic Regression: Linear Classification Boundary",
    fontsize=16,
    pad=15
)

ax.legend(title="Penguin Species")
ax.grid(alpha=0.2)

plt.tight_layout()

# Save high-resolution output
plt.savefig(
    "logistic_regression_boundary.png",
    dpi=300,
    bbox_inches="tight",
    facecolor="white"
)

plt.show()

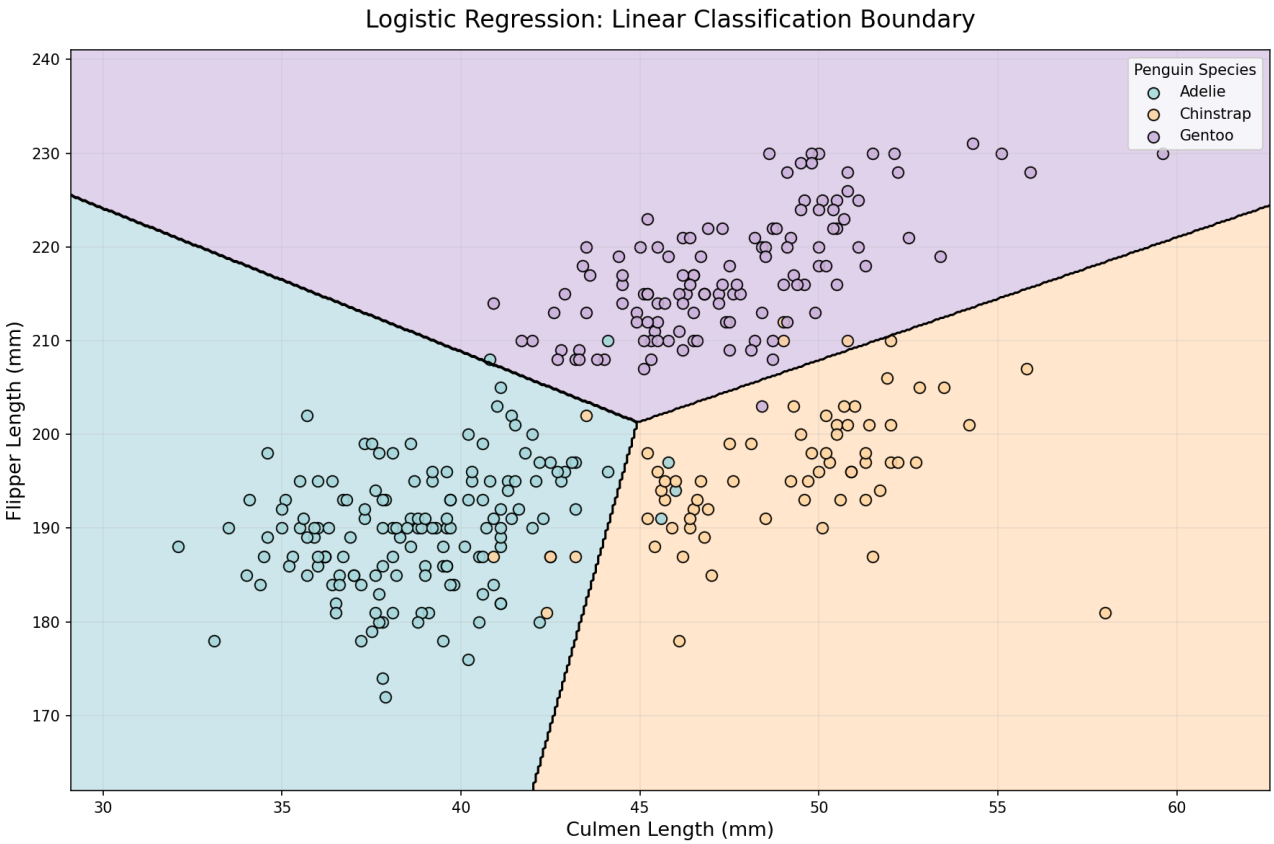
print("\nDecision boundary saved as logistic_regression_boundary.png")
```

```
========== LOGISTIC REGRESSION ==========
Accuracy: 97.10%

Classification Report:
              precision    recall  f1-score   support

      Adelie       0.97      0.97      0.97        30
   Chinstrap       1.00      0.93      0.96        14
      Gentoo       0.96      1.00      0.98        25

    accuracy                           0.97        69
   macro avg       0.98      0.97      0.97        69
weighted avg       0.97      0.97      0.97        69
```




```
Decision boundary saved as logistic_regression_boundary.png
```

Task 8: Compare CART and Normal Linear Boundary

- Plot both the Decision Tree (axis-aligned non-linear) and Logistic Regression (linear) decision boundaries on side-by-side subplots. * Compare how non-linear CART boundaries differ from straight linear hyperplanes.

In [ ]:

```
# ============================================================
# TASK 8: COMPARE CART AND NORMAL LINEAR BOUNDARY
# Decision Tree vs Logistic Regression
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# ------------------------------------------------------------
# 1. LOAD DATASET
# ------------------------------------------------------------

df = sns.load_dataset("penguins")

# Rename feature to match the assignment
df = df.rename(columns={
    "bill_length_mm": "culmen_length_mm"
})

features = ["culmen_length_mm", "flipper_length_mm"]
target = "species"

data = df[features + [target]].dropna(subset=[target]).copy()

X = data[features]
y = data[target]

# ------------------------------------------------------------
# 2. SPLIT DATA
# ------------------------------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# ------------------------------------------------------------
# 3. TRAIN CART DECISION TREE
# ------------------------------------------------------------

cart_model = make_pipeline(
```

```
    SimpleImputer(strategy="median"),
    DecisionTreeClassifier(
        criterion="gini",
        max_depth=4,
        min_samples_leaf=5,
        random_state=42
    )
)

cart_model.fit(X_train, y_train)

# ------------------------------------------------------------
# 4. TRAIN LOGISTIC REGRESSION
# ------------------------------------------------------------

linear_model = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler(),
    LogisticRegression(max_iter=2000)
)

linear_model.fit(X_train, y_train)

# ------------------------------------------------------------
# 5. EVALUATE BOTH MODELS
# ------------------------------------------------------------

cart_pred = cart_model.predict(X_test)
linear_pred = linear_model.predict(X_test)

print("========== MODEL COMPARISON ==========")
print(f"CART Decision Tree Accuracy: "
      f"{accuracy_score(y_test, cart_pred) * 100:.2f}%")

print(f"Logistic Regression Accuracy: "
      f"{accuracy_score(y_test, linear_pred) * 100:.2f}%")

# ------------------------------------------------------------
# 6. CREATE GRID FOR DECISION BOUNDARIES
# ------------------------------------------------------------

x_min = X[features[0]].min() - 3
x_max = X[features[0]].max() + 3

y_min = X[features[1]].min() - 10
y_max = X[features[1]].max() + 10

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 500),
    np.linspace(y_min, y_max, 500)
)

grid = pd.DataFrame({
    features[0]: xx.ravel(),
```

```
    features[1]: yy.ravel()
})

# Predict classes across the grid
cart_grid_pred = cart_model.predict(grid)
linear_grid_pred = linear_model.predict(grid)

class_names = list(cart_model.classes_)
class_map = {name: i for i, name in enumerate(class_names)}

Z_cart = np.array([
    class_map[c] for c in cart_grid_pred
]).reshape(xx.shape)

Z_linear = np.array([
    class_map[c] for c in linear_grid_pred
]).reshape(xx.shape)

# ------------------------------------------------------------
# 7. PLOT SIDE-BY-SIDE DECISION BOUNDARIES
# ------------------------------------------------------------

colors = ["#A8DADC", "#FFD6A5", "#CDB4DB"]

fig, axes = plt.subplots(
    1, 2,
    figsize=(17, 7),
    dpi=150,
    sharex=True,
    sharey=True
)

# ----- LEFT: CART DECISION TREE -----

axes[0].contourf(
    xx, yy, Z_cart,
    levels=np.arange(len(class_names) + 1) - 0.5,
    colors=colors[:len(class_names)],
    alpha=0.6
)

axes[0].contour(
    xx, yy, Z_cart,
    levels=np.arange(len(class_names) - 1) + 0.5,
    colors="black",
    linewidths=1.4
)

for i, species in enumerate(class_names):
    mask = y == species

    axes[0].scatter(
        X.loc[mask, features[0]],
        X.loc[mask, features[1]],
```

```
        color=colors[i],
        edgecolors="black",
        s=35,
        alpha=0.9,
        label=species
    )

axes[0].set_title(
    "CART Decision Tree\n(Axis-Aligned Non-Linear Boundaries)",
    fontsize=13
)

# ----- RIGHT: LOGISTIC REGRESSION -----

axes[1].contourf(
    xx, yy, Z_linear,
    levels=np.arange(len(class_names) + 1) - 0.5,
    colors=colors[:len(class_names)],
    alpha=0.6
)

axes[1].contour(
    xx, yy, Z_linear,
    levels=np.arange(len(class_names) - 1) + 0.5,
    colors="black",
    linewidths=1.4
)

for i, species in enumerate(class_names):
    mask = y == species

    axes[1].scatter(
        X.loc[mask, features[0]],
        X.loc[mask, features[1]],
        color=colors[i],
        edgecolors="black",
        s=35,
        alpha=0.9,
        label=species
    )

axes[1].set_title(
    "Logistic Regression\n(Linear Decision Boundaries)",
    fontsize=13
)

# ----- LABELS AND FORMATTING -----

axes[0].set_xlabel("Culmen Length (mm)", fontsize=12)
axes[1].set_xlabel("Culmen Length (mm)", fontsize=12)
axes[0].set_ylabel("Flipper Length (mm)", fontsize=12)

for ax in axes:
    ax.set_xlim(x_min, x_max)
```

```
    ax.set_ylim(y_min, y_max)
    ax.grid(alpha=0.15)

handles, labels = axes[1].get_legend_handles_labels()

fig.legend(
    handles,
    labels,
    title="Penguin Species",
    loc="lower center",
    ncol=len(class_names),
    bbox_to_anchor=(0.5, -0.02)
)

fig.suptitle(
    "Comparison of CART and Logistic Regression Decision Boundaries",
    fontsize=16,
    y=1.02
)

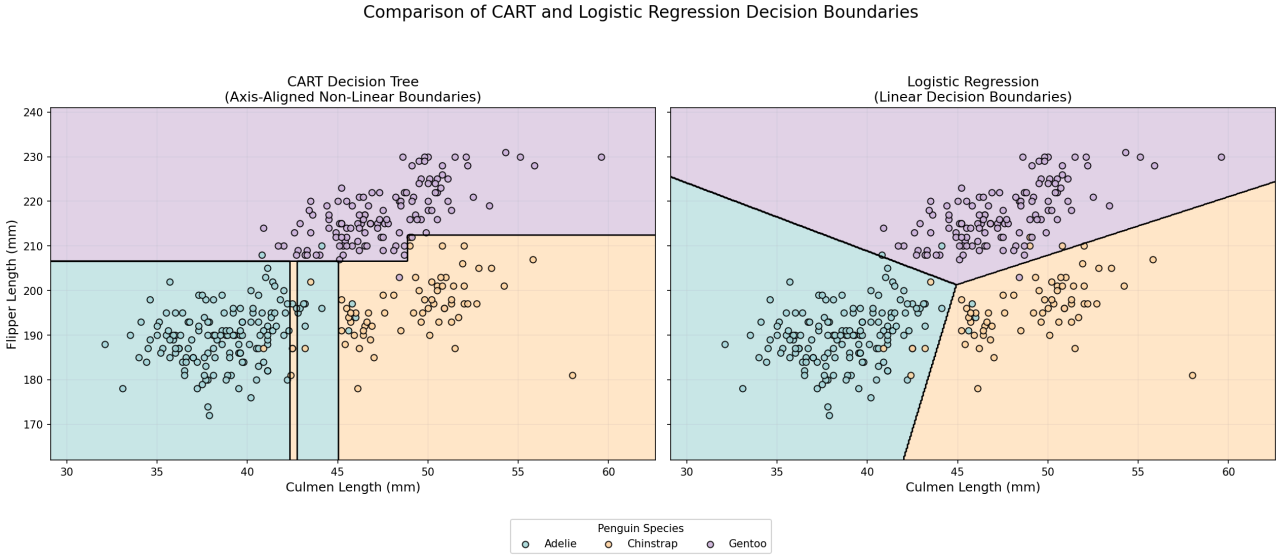
plt.tight_layout(rect=[0, 0.08, 1, 0.96])

plt.savefig(
    "cart_vs_logistic_boundary.png",
    dpi=300,
    bbox_inches="tight",
    facecolor="white"
)

plt.show()

print("\nComparison plot saved as cart_vs_logistic_boundary.png")
```

```
========== MODEL COMPARISON ==========
CART Decision Tree Accuracy: 97.10%
Logistic Regression Accuracy: 97.10%
```




```
Comparison plot saved as cart_vs_logistic_boundary.png
```

Task 9: Compare Performance

- Compare the evaluation metrics (Accuracy, Confusion Matrix, Classification Report) of the CART Classifier vs. Logistic Regression.

Conclusion

The experiment successfully demonstrated the implementation of the Classification and Regression Tree (CART) algorithm using the Palmer Penguins Extended dataset. The dataset was preprocessed to handle missing values before training. The decision tree structure was visualized, showcasing the hierarchical splits based on feature thresholds. A comparison with a normal linear boundary (Logistic Regression) highlighted CART's ability to create non-linear, axis-aligned decision boundaries to separate different penguin species.